

Vibe code app 8 (BAS Lite)

Generated by scripts/build_docs_pdf.py

April 13, 2026

- 1 Boss Pi Bas Lite Docker
- 2 Boss Pi — BAS Lite without VOLTTRON (Docker + easy-aso + Caddy)
 - 2.1 1. Install Docker Engine on Raspberry Pi OS
 - 2.1.1 1.1 Update the system
 - 2.1.2 1.2 Install Docker (official convenience script)
 - 2.1.3 1.3 Install Compose plugin (often bundled)
 - 2.1.4 1.4 Quick test
 - 2.2 2. BACnet JSON-RPC (diy-bacnet-server)
 - 2.3 3. Copy this folder to the Pi
 - 2.4 4. Build and start BAS Lite
 - 2.4.1 4.1 FastAPI OpenAPI (operator debugging)
 - 2.5 5. SD card wear — logs and Docker writes
 - 2.5.1 5.1 Compose logging (already set)
 - 2.5.2 5.2 Prune periodically
 - 2.5.3 5.3 SQLite data volume
 - 2.6 6. Security model (OT LAN)
 - 2.7 7. Related easy-aso documentation
 - 2.8 8. Developer loop on a PC (optional)
 - 2.9 9. Where did VOLTTRON go?
 - 2.10 10. Troubleshooting
 - 2.11 11. “As bad-ass as VOLTTRON?”
- 3 Bas Lite App8 Tutorial
- 4 BAS Lite on VOLTTRON — App 8 tutorial (human + AI)
 - 4.1 Quick context (for AI assistants)
 - 4.2 1. Relationship to App 7
 - 4.2.1 1.1 App 8 is an **enhanced upgrade** — not a 1:1 folder copy

- 4.3 2. Preview the React UI before building static files
 - 4.4 3. Build and deploy
 - 4.5 4. Agent configuration (app8_web_agent/config)
 - 4.5.1 Supervisory OAT share agent (oat_share_agent)
 - 4.6 5. REST surface (selected)
 - 4.7 6. BACnet + Platform Driver cheat sheet (edge)
 - 4.7.1 BACnet utilities
 - 4.7.2 Scan
 - 4.7.3 Bulk scrape configs
 - 4.7.4 Platform Driver config store (patterns)
 - 4.7.5 Proxy vs standalone scripts
 - 4.7.6 Weak devices
 - 4.8 7. Caddy on port 80 — pretty URL, Basic Auth, optional TLS
 - 4.8.1 What the config does
 - 4.8.2 Install on the Pi (summary)
 - 4.8.3 Set Basic Auth at boot
 - 4.8.4 Point at App 7 instead of App 8
 - 4.8.5 Self-signed HTTPS
 - 4.8.6 Firewall
 - 4.9 8. Pi operations: Python, storage, logging, SD card, and systemd
 - 4.9.1 8.1 Python version vs VOLTTRON
 - 4.9.2 8.2 Local storage posture (Pi)
 - 4.9.3 8.3 Logging and SD card wear
 - 4.9.4 8.4 systemd: never mix service mode and manual debug
 - 4.9.5 8.5 Example volttron.service (journal + /run log, SD-friendly)
 - 4.9.6 8.6 BACnet bring-up ladder (App 8)
 - 4.10 9. OpenClaw
- 5 Replicate Any Building Lan Topology
- 6 Recreating BAS Lite (App 8) on any HVAC OT LAN
- 6.1 1. What must stay aligned
 - 6.2 2. Platform Driver device identities
 - 6.3 3. Site model (no Python edits for a new building)
 - 6.4 4. OAT share and other small agents
 - 6.5 5. Deploy scripts from a Windows workstation
 - 6.6 6. Checklist before go-live
 - 6.7 7. Non-BACnet drivers

1 Boss Pi Bas Lite Docker

2 Boss Pi — BAS Lite without VOLTTRON (Docker + easy- aso + Caddy)

This tutorial is for **beginners** who want the **React “BAS Lite” operator UI** on a Raspberry Pi (**Pi 5 or older 3B+**) using:

- **Docker Compose** (no systemd units for the app — only Docker’s restart policy).
- **easy-aso** on PyPI: **asyncio supervisor** + **FastAPI** (/api/v1/*) and legacy JSON for the SPA (/app8/api/*).
- **diy-bacnet-server** as a **sibling** Compose service (BACnet **UDP 47808** + JSON-RPC on **8080**), with optional **Bearer** RPC auth via **BACNET_RPC_API_KEY** (same secret on **diy-bacnet** and **api**).
- **Caddy** on **80 → 443 redirect**, **TLS internal** (self-signed), **Basic Auth** for the operator UI, and a **gateway-only header** so the API is not meant to be called without going through Caddy (see **.env.example**).
- **nginx** serves the static React build under **/app8/**.

The old **VOLTTRON + app8_web_agent** tree under **volttron_data/** is **archived reference only** — see **volttron_data/ARCHIVE_NOTE.md**.

2.1 1. Install Docker Engine on Raspberry Pi OS

These steps work on **64-bit Raspberry Pi OS (Bookworm)** on **Pi 5** and **Pi 3B+** (ARM). Use a **64-bit** OS image where possible.

2.1.1 1.1 Update the system

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot    # optional but recommended after a large upgrade
```

2.1.2 1.2 Install Docker (official convenience script)

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"
```

Log out and back in so the **docker** group applies.

2.1.3 1.3 Install Compose plugin (often bundled)

```
docker compose version
```

If missing:

```
sudo apt install -y docker-compose-plugin
```

2.1.4 1.4 Quick test

```
docker run --rm hello-world
```

2.2 2. BACnet JSON-RPC (diy-bacnet-server)

easy-aso's **BACnet JSON-RPC driver** talks to **diy-bacnet-server** (or your own JSON-RPC BACnet gateway).

This stack **builds diy-bacnet** from `docker/diy-bacnet/Dockerfile`, which **shallow-clones** `diy-bacnet-server` at build time (GitHub's **default branch**). To pin a revision for reproducible OT builds, vendor a checkout or adjust that Dockerfile. The **api** service defaults to:

```
SUPERVISOR_BACNET_RPC_URL=http://diy-bacnet:8080
```

For **OT**, set a long random **BACNET_RPC_API_KEY** in `.env`. The same value is passed to **diy-bacnet** (RPC Bearer enforcement) and **api** (easy-aso's `JsonRpcBacnetClient` sends `Authorization: Bearer ...`). Leave it empty only for quick lab setups.

Ports: BACnet **UDP 47808** is published for field devices. HTTP **8080** is mapped to **127.0.0.1** on the host by default so JSON-RPC is not accidentally exposed on the whole LAN; edit `docker-compose.yml` if you need another machine to reach the RPC port directly.

If you run `diy-bacnet` **elsewhere** instead, remove or disable the **diy-bacnet** service and set **SUPERVISOR_BACNET_RPC_URL** (and the same **BACNET_RPC_API_KEY**) accordingly. `host.docker.internal` remains available on the **api** container for host-published RPC.

2.3 3. Copy this folder to the Pi

From your laptop (example):

```
scp -r vibe_code_apps_8 ben@192.168.204.12:~/bas-lite
ssh ben@192.168.204.12
cd ~/bas-lite
```

2.4 4. Build and start BAS Lite

```
cd ~/bas-lite    # folder that contains docker-compose.yml
cp .env.example .env
# Edit .env: set BAS_LITE_GATEWAY_TOKEN, BACNET_RPC_API_KEY, and a
#               new BAS_AUTH_HASH for your operator password.
docker compose build
```

```
docker compose up -d
docker compose ps
docker compose logs -f caddy
```

The **api** image installs **easy-aso** from **PyPI** via **pip install .** in `docker/bas_lite_api/Dockerfile` (declared in `docker/bas_lite_api/pyproject.toml`). **JSON-RPC Bearer** (`BACNET_RPC_API_KEY`) requires **easy-aso** $\geq 0.1.5$; rebuild after that version appears on PyPI so the resolver picks it up. If you are still on **0.1.3** from PyPI, leave `BACNET_RPC_API_KEY` unset unless `diy-bacnet` is also running without RPC auth.

Open a browser:

`https://<pi-ip>/` (port **443**). Your browser will warn about **Caddy's internal CA** — trust it on the OT workstation or install Caddy's local root (see [Caddy tls internal](#)). You will be prompted for **Basic Auth** (operator credentials from `.env`). HTTP on port **80** redirects to HTTPS.

2.4.1 4.1 FastAPI OpenAPI (operator debugging)

Caddy exposes `/openapi.json` for the machine-readable spec. The React UI already uses the path `/docs` for in-app BACnet notes, so **Swagger UI is not mounted on /docs through Caddy**.

To browse **Swagger** (`/docs` on the API container), port-forward from your laptop. If `BAS_LITE_GATEWAY_TOKEN` is set on **api**, direct calls to the container must send header `X-Bas-Lite-Gateway-Token: Bearer <same value>` (except `GET /api/v1/health`). For a simple lab, temporarily unset `BAS_LITE_GATEWAY_TOKEN` on **api** while using SSH forward only.

```
ssh -L 8090:127.0.0.1:8090 ben@PI_IP
# then open http://127.0.0.1:8090/docs
```

REST CRUD for devices/points: `/api/v1/*` (see [easy-aso supervisor docs](#)).

2.5 5. SD card wear — logs and Docker writes

Pi SD cards fail faster with **constant logging to the card**.

2.5.1 5.1 Compose logging (already set)

`docker-compose.yml` uses the **local** log driver with **size caps** for **api**, **frontend**, and **caddy**. That avoids huge JSON log files on disk.

2.5.2 5.2 Prune periodically

```
docker system prune -f
```

2.5.3 5.3 SQLite data volume

Supervisor state lives in Docker volume **bas_lite_data** mounted at **/data** in the API container (`supervisor.sqlite`, `schedule JSON`, `driver config files`). That is **bounded** application state — not high-churn logs.

2.6 6. Security model (OT LAN)

This is **not** internet hardening (no rate limits). It **is** meant for a **segmented OT / BAS LAN**:

Layer	What it does
Caddy <code>tls internal</code>	HTTPS with a self-signed site cert (trust Caddy's local CA or accept the browser warning).
Basic Auth	Operator login before the SPA or API paths load through Caddy (<code>BAS_AUTH_USER</code> + <code>BAS_AUTH_HASH</code> bcrypt in <code>.env</code>).
<code>BAS_LITE_GATEWAY_TOKEN</code>	Caddy injects X-Bas-Lite-Gateway-Token: Bearer ... on <code>/app8/api/*</code> , <code>/api/v1/*</code> , and <code>/openapi.json</code> only. The api container checks it when the env var is set. The

Layer	What it does
	browser never holds this secret. Caddy drops any client-supplied value with the same header name before proxying.
BACNET_RPC_API_KEY	diy-bacnet-server's JSON-RPC Bearer auth; easy-aso sends the same header when calling RPC.

Do not publish the **api** container port **8090** to the host in production; only **Caddy** should be reachable on **80/443**. Keep **SQLite** and driver data on the named volume.

Lab without TLS: set **CADDYFILE=docker/caddy/Caddyfile.local-dev** in **.env** and leave **BAS_LITE_GATEWAY_TOKEN** empty on **api**. That stack uses HTTP on port **80** only (no Basic Auth in Caddy — use only on an isolated lab network).

2.7 7. Related easy-aso documentation

Supervisor workflows and BACnet drivers:

SUPERVISOR_WORKFLOWS. OT-oriented notes on RPC secrets and exposure: **Supervisor OT security**.

2.8 8. Developer loop on a PC (optional)

```
cd frontend
$env:VITE_DEV_PROXY_TARGET="http://192.168.204.12"
npm install
npm run dev
```

vite.config.ts proxies /app8/api to the Pi API.

2.9 9. Where did VOLTTRON go?

Supervisory polling, device/point config, and HTTP APIs are handled by **easy-aso** inside the **api** container. The React app keeps the same **/app8/api/*** JSON shape for pages like Overview, Live points, and Trends.

If you need the **old bench** (Platform Driver + vctl), it remains under **vibe_code_apps_6 / archived volttron_data/** trees for reference — not used by this Docker stack.

2.10 10. Troubleshooting

Symptom	Check
UI loads but no BACnet values	<code>docker compose logs api</code> — RPC URL / BACNET_RPC_API_KEY mismatch? From the host: <code>curl -sS http://127.0.0.1:8080/</code> should return 200 (diy-bacnet maps 8080 to localhost).
502 from Caddy	<code>docker compose ps</code> — is api healthy? <code>docker compose logs api</code> .
401 on API through Caddy	BAS_LITE_GATEWAY_TOKEN must match on caddy and api ; Caddy must use the default Caddyfile (not local-dev) when the token is set.
Caddy will not start — port 443 in use	Common on Windows (IIS, Hyper-V, other VPN or proxy tools). Either stop the other listener or change the compose mapping to e.g. 8443:443 under caddy.ports and browse <code>https://<host>:8443</code> .
Cannot write setpoint	

Symptom	Check
	Driver must be bacnet_jsonrpc for that device; confirm diy-bacnet write RPC works.
Out of disk	docker system df, prune images, shrink log files.

2.11 11. “As bad-ass as VOLTTRON?”

You get **asyncio**, **typed FastAPI**, **SQLite-backed** config, **hot reload** of drivers, **Docker-first** packaging, **Caddy** edge routing, and **PyPI-installable** core (easy-aso). What you **do not** get out of the box is VOLTTRON’s full **VIP bus**, **historian ecosystem**, and **years of BACnet proxy edge cases** — trade intentionally for a **smaller** modern stack you can own.

When you outgrow the shim, point the React app at **/api/v1** directly and drop legacy **/app8/api** routes.

3 Bas Lite App8 Tutorial

4 BAS Lite on VOLTTRON — App 8 tutorial (human + AI)

This document is the **upgraded, self-contained** App 8 handoff: same **edge BACnet** story as App 7, but the UI is the **compiled React** app under **/app8/**, the web agent adds **operator APIs** (Pi metrics, vctl, driver config store, weekly occupancy schedule), and **\$8** carries the full **systemd + logging + SD card** operations that used to live only in the App 7 bosspi tutorial—so **App 8 = enhanced stack + enhanced tutorial** in one place.

4.1 Quick context (for AI assistants)

Item	Typical value
Agent VIP	ben.app8.web
Install tag	ben-app8-web
HTTP prefix	/app8
UI (direct VOLTTRON)	http://<pi>:8080/app8/index.html
UI (via Caddy, optional)	http://<pi>/ → redirects into /app8/
VOLTTRON root (Pi)	/home/ben/volttron
VOLTTRON_HOME	/home/ben/.volttron
Agent path on Pi	/home/ben/volttron/volttron_data/ben_bacnet/app8_web_agent
BACnet bench devices	BensFakeAHU, Zone1VAV (override via agent config → bacnet_devices)
OAT share agent (optional)	ben.oat.share / tag ben-oat-share — 15 min read + writes (see §4)

4.2 1. Relationship to App 7

- **Same class of integration:** BACnet proxy + Platform Driver topics → web agent → browser.
- **App 7** served a small static/vanilla bundle under /app7.
- **App 8** serves a **production React build** from app8_web_agent/webroot/ with richer pages: driver config tree, vctl table, occupancy editor, trends, faults.

4.2.1 1.1 App 8 is an enhanced upgrade — not a 1:1 folder copy

vibe_code_apps_8 does not duplicate every file from **vibe_code_apps_7**. It **inherits the same bench idea** (BACnet, Platform Driver, Pi edge) and **replaces / extends** the pieces that needed to level up:

Topic	App 7 folder	App 8 folder
Operator web	app7_web_agent + hand-written webroot/app7/*	app8_web_agent + Vite/React frontend/ → built webroot/ Same BACnet semantics , plus Pi metrics, vctl, driver config store, occupancy schedule APIs
Supervisory patterns	(none in repo)	oat_share_agent (e.g. OAT read / share on 15 min cadence)
Port 80 / TLS / Basic Auth	Discussed in passing	caddy/ pack + §7 below This document §8 now inlines the systemd + SD + logging material so App 8 is self-contained ; App 7's doc stays the reference for App 7-only paths and history
Edge ops bible	bosspi-volttron-dataflow-and-storage-tutorial.md (deep)	

So: **conceptually** App 8 is “App 7 + React + operator tooling + Caddy + OAT share”; **physically** it is a **sibling project** with **new and rewritten** trees, not a tarball copy of **vibe_code_apps_7**.

4.3 2. Preview the React UI before building static files

On a dev machine (PowerShell):

```
cd vibe_code_apps_8\frontend
npm install
$env:VITE_DEV_PROXY_TARGET="http://192.168.204.12:8080"
npm run dev
```

Open **http://localhost:5173/app8/** (note the **/app8/** path — it matches the Vite base). The dev server proxies **/app8/api/*** to the Pi so you see live BACnet data. When satisfied, run **npm run build** to emit static files into the agent webroot/.

4.4 3. Build and deploy

On the dev machine:

```
cd vibe_code_apps_8\frontend
npm install
npm run build
```

Deploy to the Pi (Windows example):

```
cd vibe_code_apps_8
.\deploy-app8-to-bosspi.ps1
```

On the Pi, confirm:

```
cd /home/ben/volttron
export VOLTTRON_HOME=/home/ben/.volttron
source env/bin/activate
vctl status
```

4.5 4. Agent configuration (app8_web_agent/config)

Important keys:

Key	Purpose
route_prefix	Default /app8 — must match Vite base.
bacnet_devices	Driver topic names for devices/<name>/all subscriptions.
volttron_root	Working directory for vctl subprocess calls.
vctl_path	Full path to vctl in the venv.
allow_agent_lifecycle	When true, UI may call vctl start/stop/restart/remove (dangerous if exposed broadly).
allow_driver_config_writes	When true, UI may vctl config store/delete for platform.driver.
site_model_path	Optional JSON file (path relative to the agent package or absolute) defining devices, points, and optional alarm_definitions so a new site does not require editing Python.
devices / points / alarm_definitions	Optional inline keys in the same config file (same schema as the JSON file); values override the file when both are present.
default_trend_point_id	Which pointId the trends API and UI use first (otherwise the first point id in sorted order).

See [docs/replicate-any-building-lan-topology.md](#) for OT LAN / BACnet / driver alignment and a deployment checklist. Example payload: volttron_data/ben_bacnet/app8_web_agent/site_model.example.json.

After editing **device** or **registry** configs in the UI, restart **platform.driver** from the System page (or `vctl restart --tag platform_driver`).

4.5.1 Supervisory OAT share agent (`oat_share_agent`)

Many BAS jobs treat **outside air temperature** as **one physical sensor** (e.g. at a boiler or utility panel) that is **read once** and **written / shared** to other controllers (AHUs, VAVs) on an interval—often **15 minutes** for slow-changing air temperature.

This repo includes `volttron_data/ben_bacnet/oat_share_agent/`:

Item	Value
VIP identity	<code>ben.oat.share</code>
Install tag	<code>ben-oat-share</code>
Default interval	900 s (15 min), configurable <code>interval_seconds</code>
First run delay	30 s after start (<code>first_sync_delay_seconds</code>) so Platform Driver is warm

Behavior each cycle

1. **get_point** on the **source** (`source_device_path`, `source_point`) — path must match the Platform Driver config-store key **without** the `devices/` prefix (e.g. store key `devices/campus/bench/ahu` → path `campus/bench/ahu`; many benches use a single segment like `BensFakeAHU`).
2. **set_point** on every **target** (`targets[].device_id`, `targets[].point_name`) — same `device_id` style as the App 8 setpoint API (driver topic name, e.g. `Zone1VAV`).

Config (`oat_share_agent/config`) ships with **targets: []** so a fresh install only **reads** and logs until you add consumer points. Each target point must exist in that device's **registry CSV** and be **writable** if the BACnet object is not intrinsically writeable.

Example target (only after you add `SharedOAT` to the consumer registry):

```
"targets": [
  {"device_id": "Zone1VAV", "point_name": "SharedOAT"}
]
```

Deploy from Windows (same pattern as App 8):

```
cd vibe_code_apps_8
.\deploy-oat-share-to-bosspi.ps1
```

Manual install on the Pi:

```
vctl install --vip-identity ben.oat.share --tag ben-oat-share \
  /home/ben/volttron/volttron_data/ben_bacnet/oat_share_agent \
  --config /home/ben/volttron/volttron_data/ben_bacnet/
    oat_share_agent/config
vctl start --tag ben-oat-share
```

4.6 5. REST surface (selected)

Method	Path	Notes
GET	/app8/api/health	Same shape as App 7 health, plus defaultTrendPointId for the trends UI/API default curve.
GET	/app8/api/points	Live point table + metadata.
POST	/app8/api/setpoints/write	BACnet write via Platform Driver RPC.
GET	/app8/api/system/metrics	Pi load, memory, root disk; optional psutil CPU %.
GET	/app8/api/agents/vctl	Raw vctl list plus parsed UUID rows.
POST	/app8/api/agents/lifecycle	JSON { "action": "start stop restart remove", "tag"?:

Method	Path	Notes
		"...", "uuid"?: "..." }.
GET	/app8/api/ driver/configs	vctl config list platform.driver.
GET	/app8/api/ driver/config? name=...	vctl config get platform.driver <name>.
POST	/app8/api/ driver/config/ store	JSON { "name", "content", "csv": bool }.
POST	/app8/api/ driver/config/ delete	JSON { "name" }.
GET/POST	/app8/api/ schedule	Weekly occupancy JSON (agent-local schedule_store.json).

4.7 6. BACnet + Platform Driver cheat sheet (edge)

Assumptions: ~/volttron clone, source env/bin/activate, Python **3.10+** for current VOLTTRON 9.x / modular stacks (pin with pip show volttron on the Pi).

4.7.1 BACnet utilities

```
cd ~/volttron && source env/bin/activate
cd scripts/bacnet
```

Edit BACpypes.ini: **address must be the edge NIC**, not the remote BACnet device.

4.7.2 Scan

```
python bacnet_scan.py --range 0 4194303 --timeout 15 --csv-out  
devices.csv
```

4.7.3 Bulk scrape configs

```
python grab_multiple_configs.py devices.csv --out-directory ./  
scan_output --ini ./BACpypes.ini
```

Produces devices/ and registry_configs/ suitable for vctl config store.

4.7.4 Platform Driver config store (patterns)

```
vctl config store platform.driver registry_configs/ahul.csv  
configs/ahul.csv --csv  
vctl config store platform.driver devices/site/building/ahul  
configs/ahul.config  
vctl config list platform.driver
```

Restart driver after changes:

```
vctl restart --tag platform_driver
```

4.7.5 Proxy vs standalone scripts

Standalone bacnet_scan.py / grab_bacnet_config.py **conflict** with a running BACnet proxy unless you use the **proxy_*** variants. For normal runtime polling, use **BACnet Proxy + Platform Driver**.

4.7.6 Weak devices

Set "use_read_multiple": false and tune "max_per_request" in BACnet driver config when devices reject RPM or segment poorly.

4.8 7. Caddy on port 80 — pretty URL, Basic Auth, optional TLS

VOLTTRON's platform web is usually `http://<pi>:8080/` with a generic `/index.html` and each web agent on its own prefix (`/app8/`, `/app7/`, ...). Operators often prefer `http://<pi>/` on **port 80** and (optionally) **HTTPS** with a login gate **outside** Python.

This repo ships a **separate systemd unit** `caddy-bas-lite` (not the stock Debian `caddy.service`) so you can opt in without silently hijacking a pre-existing Caddy install until you disable the default unit.

4.8.1 What the config does

1. **Exact root redirect** — GET `/` → **302** to `{CADDY_APP_PREFIX}/index.html` (default `/app8/index.html`). That sends humans straight into the React shell instead of VOLTTRON's default landing page.
2. **Reverse proxy** — everything else is `reverse_proxy` to `127.0.0.1:8080`, so `/app8/assets/...`, `/app8/api/health`, etc. are served by VOLTTRON unchanged. Caddy adds **X-Forwarded-Proto**, **X-Forwarded-For**, and **X-Forwarded-Host** so relative URLs and APIs behave correctly through the proxy.
3. **Optional Basic Auth** — when `CADDY_BASIC_AUTH_ENABLE=1`, Caddy's `basic_auth` runs before the redirect/proxy. The password is a **bcrypt** hash from the Caddy CLI (never store plaintext on disk).
4. **Optional TLS** — when `CADDY_TLS_ENABLE=1`, **port 80** only issues a **permanent redirect to HTTPS**, and **port 443** terminates TLS using `CADDY_TLS_CERT` / `CADDY_TLS_KEY` (typically **self-signed** for bench LANs).

Packaged files live under `vibe_code_apps_8/caddy/` — see `caddy/README.md` for a file table.

4.8.2 Install on the Pi (summary)

```
sudo bash vibe_code_apps_8/caddy/scripts/install-caddy-bas-lite.sh
sudo nano /etc/default/caddy-bas-lite
```

```
sudo bas-lite-render-caddyfile.sh
sudo caddy validate --config /etc/caddy/bas-lite.caddy --adapter
caddyfile
sudo systemctl enable --now caddy-bas-lite
```

Boot-time behavior is controlled entirely by `/etc/default/caddy-bas-lite` (copied from `caddy/env.example`). The `ExecStartPre` in `caddy-bas-lite.service` re-renders the Caddyfile on every start so you can bake the same image and only change the env file per site.

4.8.3 Set Basic Auth at boot

1. On the Pi (or any machine with the caddy binary):

```
caddy hash-password --plaintext 'YourLongRandomPassword'
```

2. In `/etc/default/caddy-bas-lite`:

- `CADDY_BASIC_AUTH_ENABLE=1`
- `CADDY_BASIC_AUTH_USER=operator` (or any username)
- `CADDY_BASIC_AUTH_HASH='<paste bcrypt line>'`

3. `sudo bas-lite-render-caddyfile.sh && sudo systemctl reload caddy-bas-lite`

4.8.4 Point at App 7 instead of App 8

Set `CADDY_APP_PREFIX=/app7` to send `/` to `/app7/index.html`. No change to VOLTTRON is required beyond having that agent installed.

4.8.5 Self-signed HTTPS

```
sudo bash vibe_code_apps_8/caddy/scripts/gen-selfsigned-cert.sh
'192.168.204.12'
```

Then enable `CADDY_TLS_ENABLE=1` and ensure `CADDY_TLS_CERT / CADDY_TLS_KEY` match the generated paths. Browsers will show a warning until you trust the cert (bench-appropriate).

4.8.6 Firewall

If `ufw` is enabled:

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

4.9 8. Pi operations: Python, storage, logging, SD card, and systemd

This section is the **operational upgrade** bundled with App 8: the same **edge discipline** as the App 7 bossPi tutorial, **inlined here** so you do not have to cross-reference for **systemd**, **journald** / **RAM logging**, or **SD wear**. The canonical long-form narrative for the bench remains `vibe_code_apps_7/docs/bossPi-volttron-dataflow-and-storage-tutorial.md`.

4.9.1 8.1 Python version vs VOLTTRON

Match Python to the **exact** stack on the Pi:

- **Modular / PyPI volttron: Python $\geq$ 3.10** (tested baseline **3.10**; **3.11** often works — confirm with `pip show volttron`).
- **Older monolithic clones (e.g. 8.x)**: follow that release’s install guide; do not assume 3.10+.

```
cd /home/ben/volttron
source env/bin/activate
python3 --version
pip show volttron | sed -n '1,12p'
```

4.9.2 8.2 Local storage posture (Pi)

- **Live BACnet / UI telemetry: memory-first and bounded** (same idea as App 7).
- **VOLTTRON_HOME (~/.volttron)**: still holds configs, keystores, agent installs, and **some** logs — “RAM only” is **not** literal for the

whole platform; avoid **high-churn** append logs on the SD card.

4.9.3 8.3 Logging and SD card wear

Logs may land in: **VOLTTRON files** under `~/volttron` or `~/.volttron`, **journald** (`volttron.service`), or **custom** agent files.

Design goals

- No **unbounded** per-sample or debug logging to the card in normal operation.
- **INFO/WARN** by default; **DEBUG** only while troubleshooting.
- **Rotate** or bound any file sink you keep on disk.

Prefer journald in RAM (strong Pi pattern) — logs lost on reboot; good for bench SD longevity:

```
sudo mkdir -p /etc/systemd/journald.conf.d
sudo tee /etc/systemd/journald.conf.d/volatile-storage.conf >/dev/
null <<'EOF'
```

```
[Journal]
```

```
Storage=volatile
```

```
RuntimeMaxUse=64M
```

```
EOF
```

```
sudo systemctl restart systemd-journald
```

If journald stays persistent, cap it:

```
[Journal]
```

```
SystemMaxUse=100M
```

```
MaxRetentionSec=1week
```

```
journalctl --disk-usage
```

VOLTTRON `volttron.log`: avoid a single ever-growing file on the card — use **rotation**, **/run** (tmpfs) for the `-l` path (see §8.5), or `stdout/stderr` → journal only.

Other SD practices: tmpfs `/tmp`, sensible **swap/zram**, **noatime** where appropriate, move heavy DBs/historians to **USB SSD** or another host; **read-only root** is advanced.

Inspect growth

```
systemctl status volttron.service --no-pager
journalctl -u volttron.service -n 100 --no-pager
ls -lah /home/ben/.volttron
find /home/ben/.volttron -type f -printf '%s %p\n' 2>/dev/null |
    sort -nr | head -30
```

4.9.4 8.4 systemd: never mix service mode and manual debug

1. **Service mode** — `systemctl start volttron.service`; use `journalctl / systemctl status`.
2. **Manual debug** — `sudo systemctl stop volttron.service` first, then activate the venv and run the platform manually.

Mixing the two causes **stale VIP sockets**, **double binds**, and confusing `vctl status`.

Common commands

```
sudo systemctl status volttron.service --no-pager
sudo systemctl stop volttron.service
sudo systemctl start volttron.service
sudo systemctl restart volttron.service
sudo systemctl enable volttron.service
journalctl -u volttron.service -n 100 --no-pager
```

4.9.5 8.5 Example volttron.service (journal + /run log, SD-friendly)

Adapt User / Group / paths. `RuntimeDirectory=volttron + -l /run/volttron/volttron.log` keeps the optional file log on **tmpfs** (cleared on reboot), not grinding the SD. `StandardOutput/StandardError` go to the journal.

[Unit]

```
Description=VOLTTRON platform (bosspi)
After=network-online.target
Wants=network-online.target
```

[Service]

```
Type=simple
User=ben
Group=ben
RuntimeDirectory=volttron
WorkingDirectory=/home/ben/volttron
Environment=VOLTTRON_HOME=/home/ben/.volttron
ExecStart=/home/ben/volttron/env/bin/volttron -vv -l /run/
             volttron/volttron.log
Restart=on-failure
RestartSec=5
```

```
LimitNOFILE=65535
```

```
StandardOutput=journal
StandardError=journal
SyslogIdentifier=volttron
```

[Install]

```
WantedBy=multi-user.target
```

Install: `sudo cp volttron.service /etc/systemd/system/`, then `sudo systemctl daemon-reload` and `sudo systemctl enable --now volttron.service`.

4.9.6 8.6 BACnet bring-up ladder (App 8)

1. Pi platform healthy (**systemd** + `vctl status`)
 2. BACnet proxy healthy
 3. Platform Driver healthy and topics publishing
 4. `/app8/api/health` and live points look sane
 5. React UI renders (direct **:8080** or via **Caddy §7**)
-

4.10 9. OpenClaw

Treat this folder plus this tutorial as **model context** for redeploy:
build React → sync `app8_web_agent` → restart `ben-app8-web` → verify `/app8/api/health` and BACnet topics.

5 Replicate Any Building Lan Topology

6 Recreating BAS Lite (App 8) on any HVAC OT LAN

This note is for operators and integrators who need the same **VOLTTRON + Platform Driver** edge stack on a different site, VLAN, or BACnet plant layout. Nothing here assumes a particular bench hostname or IP range.

6.1 1. What must stay aligned

Layer	What to configure
Network	Edge host can reach field controllers on the BACnet / OT VLAN (routing, ACLs, BBMD if used).
BACnet	BACpypes.ini (or stack equivalent), UDP 47808 bind interface, BBMD table if applicable.
Platform Driver	One registry CSV (or driver-specific config) per device identity; device names must match what the agent subscribes to.
App 8 agent	bacnet_devices, optional site_model_path / inline devices + points, alarms, site_name, volttron_root, vctl_path.
Reverse proxy	Caddy (or other) upstream URL, Basic Auth secrets, TLS policy. See vibe_code_apps_8/caddy/README.md.

6.2 2. Platform Driver device identities

The web agent subscribes to `devices/<device_id>/all` for each id in `bacnet_devices` in `app8_web_agent/config`.

- Use **stable string identities** that match your driver config (often the same as the CSV/registry folder name under `devices/`).
- Prefer a naming convention that scales: `campus/building/ahu_01` or `SITE_AHU01` — avoid embedding IP addresses in the identity unless that is already your standard.

6.3 3. Site model (no Python edits for a new building)

You can describe equipment and points in JSON instead of editing `agent.py`.

Option A — external file (next to the agent package or absolute path):

```
{  
  "site_model_path": "site_model.json"  
}
```

Option B — inline in config (same schema as the file): top-level keys `devices`, `points`, optional `alarm_definitions`.

Schema summary:

- **devices**: object keyed by device id; each value may include `displayName`, `kind`, `address`, `BACnet deviceId`, `pollingEnabled`, etc. Point live values still arrive via VOLTTRON publish; `points` starts as `{}`.
- **points**: array of objects with `pointId`, `deviceId`, `name` (driver point name), `label`, `units`, `kind` (`analog` | `binary`), `adjustable`, optional `graphicGroup`.
- **alarm_definitions**: array of rules. Supported `conditionType` values in the agent today: `greaterThanSetpointPlusOffset` (needs `referencePointId` and `offset`), `boolFalse`.

Copy `volttron_data/ben_bacnet/app8_web_agent/site_model.example.json` as a starting point and align names with your Platform Driver registries.

Optional config keys:

- **default_trend_point_id**: first trend shown in the UI / API default.
- **bacnet_devices**: overrides the default list taken from all device keys in the site model (useful if the edge should only subscribe to a subset).

6.4 4. OAT share and other small agents

Any agent that references device or point names must use the **same identities** as Platform Driver. After renaming devices, update `oat_share_agent` config (or equivalent) so source and target device ids and point names match the live plant.

6.5 5. Deploy scripts from a Windows workstation

`deploy-app8-to-bosspi.ps1` and `deploy-oat-share-to-bosspi.ps1` accept parameters so you can target another host or tree without editing the file:

```
.\deploy-app8-to-bosspi.ps1 -SshTarget 'user@10.0.0.50' -  
RemoteVolttronRoot '/home/user/volttron'
```

Defaults match the original bench layout.

6.6 6. Checklist before go-live

1. From the edge shell: `vctl status` — Platform Driver and `ben.app8.web` (or your identity) **running**.
2. `curl -sS http://127.0.0.1:8080/app8/api/health` (adjust port/prefix) — `counts.devices / points` non-zero when the model is loaded.

3. BACnet **who-is** / **read** from the same interface the driver uses (firewall and binding).
4. Journald / SD card: follow Pi logging guidance in docs/bas-lite-app8-tutorial.md (§8).

6.7 7. Non-BACnet drivers

The UI and site model are **driver-agnostic**: anything exposed through the Platform Driver RPC and publish contract can use the same devices / points metadata. BACnet is the reference implementation in this repo; Modbus, SNMP, or custom drivers follow the same pattern as long as device ids and point names match.